NeonHeap: An Interactive Memory Allocation Visualizer with Live Operating System Process Monitoring

Mehak Jain, Anuj Kumar Singh

Rishihood University

mehak.j23csai@nst.rishihood.edu.in, anuj.k23csai@nst.rishihood.edu.in

***Abstract*—**

This paper presents NeonHeap, a full-stack memory allocation visualizer that combines a C backend with a React frontend to demonstrate core operating systems concepts interactively. The system implements three contiguous memory allocation algorithms—First Fit, Best Fit, and Worst Fit—along with memory compaction and a buddy system allocator, all backed by real OS memory via `mmap()`. A unique Live System Mode allows users to observe real running processes on macOS and Linux, rendered as a cyberpunk city skyline where building height is proportional to resident memory. Two new features —process detail inspection on click and a system-wide memory pressure indicator—extend the live mode with data retrieved through platform-specific system calls (`proc_pidinfo` on macOS, `/proc` filesystem on Linux). The educational mode provides algorithm comparison, fragmentation analysis, and step-by-step timeline scrubbing. The C backend exposes a JSON API over POSIX sockets, enabling the React frontend to operate as a standalone client. Experimental results demonstrate the tool's effectiveness in illustrating how different allocation strategies affect fragmentation and memory utilization under varied workloads.

***Keywords*—** Memory Allocation, Operating Systems, First Fit, Best Fit, Worst Fit, Buddy System, Memory Compaction, Process Visualization, React, mmap

I. Introduction

Memory management is a fundamental responsibility of modern operating systems. The OS must allocate, track, and reclaim memory for dozens to thousands of concurrent processes while minimizing wasted space due to fragmentation. Despite its importance, memory management remains one of the most abstract topics for students, as the internal state of the memory allocator is typically invisible during normal system operation.

Existing educational tools often present allocation algorithms as static diagrams or command-line simulations that fail to convey the dynamic, real-time nature of memory management. Furthermore, few tools bridge the gap between simulated behavior and actual operating system activity by displaying live process data alongside educational simulations.

This paper presents NeonHeap, a Memory Allocation Visualizer that addresses these limitations through two key contributions:

- **Educational Mode:** An interactive simulation of First Fit, Best Fit, Worst Fit, compaction, and buddy system algorithms, with real-time visualization, algorithm comparison, and fragmentation analysis.
- **Live System Mode:** A real-time display of actual OS processes with process detail inspection and system-wide memory pressure monitoring, using platform-specific system calls.

The system uses a C backend that allocates real memory via `mmap()` and exposes a JSON API over POSIX sockets, consumed by a React frontend rendered as a cyberpunk-themed city skyline. This paper describes the architecture, algorithms, implementation, and results.

II. Related Work

Several tools have been developed to teach memory management concepts. Stallings [1] provides comprehensive coverage of memory allocation algorithms in textbook form but lacks interactive visualization. The OS-Sim project [2] offers a command-line simulator for process scheduling and memory allocation but does not connect to real OS data. Web-based tools such as those described by Robbins [3] provide algorithm animations but are limited to predefined scenarios without live system integration.

The `top` and `htop` utilities [4] display real process information but do not visualize allocation algorithms or fragmentation concepts. Tools like Valgrind [5] focus on memory debugging rather than educational visualization of allocation strategies.

NeonHeap distinguishes itself by (a) combining simulated and real OS data in a single interface, (b) using actual `mmap()`-backed memory for the educational simulation, (c) providing platform-specific process introspection through `proc_pidinfo()` on macOS and the `/proc` filesystem on Linux, and (d) rendering the output as an engaging visual metaphor (city skyline) rather than tables or text.

III. Methodology

A. System Architecture

NeonHeap follows a client-server architecture. The C backend serves two roles: (i) a memory manager that maintains a linked list of allocated and free blocks backed by `mmap()`, and (ii) an HTTP server that exposes the memory state as JSON endpoints over POSIX sockets. The React frontend consumes these endpoints and renders the visualization.

TABLE I
System Components

Component	Technology	Purpose
Memory Manager	C (linked list)	Allocation, deallocation, compaction, buddy system
OS Memory Layer	mmap/munmap, sysctl	Real virtual memory backing
Process Reader	libproc (macOS), /proc (Linux)	Live process scanning and detail
HTTP Server	POSIX sockets	JSON API for frontend
Frontend	React 18 + Vite	Interactive UI rendering

B. Memory Allocation Algorithms

The memory manager maintains a singly linked list of `MemoryBlock` structures. Each block stores a start address, size, process ID, and a boolean flag indicating whether it is a hole (free) or allocated. The three implemented algorithms scan this list differently:

- **First Fit:** Traverses the list and selects the first hole whose size ≥ requested size. Time complexity: O(n).
- **Best Fit:** Scans all holes and selects the smallest hole that fits. Minimizes leftover space but creates small unusable fragments. Time complexity: O(n).
- **Worst Fit:** Selects the largest available hole. Leaves larger remaining holes but depletes the biggest free regions first. Time complexity: O(n).

C. Memory Compaction

Compaction relocates all allocated blocks to one end of memory, coalescing all free space into a single large hole. This eliminates external fragmentation at the cost of moving processes in memory. The implementation slides each allocated block to the lowest available address and updates the linked list accordingly.

D. Buddy System

The buddy system divides memory into power-of-2 sized blocks. When a request arrives, the smallest power-of-2 block that fits is found. If no block of that size exists, a larger block is recursively split in half until the desired size is reached. On deallocation, the freed block is merged with its "buddy" (the adjacent block of the same size) if the buddy is also free. This approach trades internal fragmentation for efficient O(log n) allocation and deallocation.

E. External Fragmentation Measurement

External fragmentation is calculated using the formula:

$$\text{Fragmentation (\%)} = (\text{Total Free Memory} - \text{Largest Hole}) / \text{User Memory} \times 100 \quad (1)$$

A fragmentation of 0% indicates all free memory is in one contiguous block. Higher values indicate scattered holes that may prevent allocation even when total free memory is sufficient.

F. Live System Mode — Process Introspection

On macOS, the process reader uses `proc_listallpids()` to enumerate all PIDs, then calls `proc_pidinfo(pid, PROC_PIDTASKALLINFO)` for each to obtain resident memory (RSS), virtual memory size, thread count, page faults, pageins, and copy-on-write faults. The executable path is retrieved via `proc_pidpath()`. On Linux, equivalent data is obtained by reading `/proc/<pid>/status`, `/proc/<pid>/stat`, and `/proc/<pid>/exe`.

G. Memory Pressure Calculation

System-wide memory pressure is computed as:

$$\text{Used (\%)} = \Sigma\, RSS_{\text{all processes}} / \text{Total RAM} \times 100 \quad (2)$$

On macOS, total RAM is obtained via `sysctl(HW_MEMSIZE)` and used RAM is the sum of RSS across all scanned processes. On Linux, `/proc/meminfo` provides `MemTotal` and `MemAvailable` directly. The pressure level is classified as: LOW (<50%), MODERATE (50–70%), HIGH (70–85%), or CRITICAL (>85%).

IV. Results and Discussion

A. Algorithm Comparison

Table II shows the fragmentation results when allocating five processes (100 KB, 200 KB, 150 KB, 50 KB, 300 KB) into a 768 KB user memory space using each algorithm.

TABLE II
Algorithm Comparison Results

Algorithm	Processes Allocated	External Fragmentation (%)	Relative Speed
First Fit	4 of 5	0.00	Fastest
Best Fit	4 of 5	0.00	Slowest
Worst Fit	4 of 5	0.00	Moderate

In this scenario, all three algorithms produce similar fragmentation because the workload fits without creating scattered holes. Differences emerge under heavier workloads with

interleaved allocations and deallocations, where Best Fit tends to create many small unusable holes and First Fit concentrates fragmentation near the beginning of memory.

B. Live Process Monitoring

The Live System Mode successfully displays the top 10 processes by RSS on macOS, refreshing every 3 seconds. Table III shows a sample output from the `/api/memory/pressure` endpoint on an 8 GB macOS system.

TABLE III
Sample Memory Pressure Output

Metric	Value
Total RAM	8.0 GB
Used RAM	4.7 GB
Used Percentage	58.9%
Pressure Level	MODERATE

C. Process Detail Inspection

Clicking a building in Live Mode fetches extended process information via the `/api/process/<pid>` endpoint. The response includes thread count, page faults, pageins, copy-on-write faults, executable path, and start time. Testing with a renderer process (PID 59935) returned 21 threads, 2,152,778 page faults, 26,640 pageins, and 2,104 COW faults — providing students with concrete insight into how a real application interacts with the virtual memory subsystem.

D. Visualization Effectiveness

The city skyline metaphor maps naturally to memory usage: taller buildings represent more memory consumption, neon colors distinguish large from small processes, and the horizontal layout mirrors the linear address space. The cyberpunk theme with particle effects and glassmorphism increases student engagement by transforming a traditionally abstract topic into a visually compelling experience.

V. Conclusion

This paper presented NeonHeap, a Memory Allocation Visualizer that bridges the gap between theoretical OS concepts and real system behavior. The tool implements First Fit, Best Fit, Worst Fit, compaction, and buddy system algorithms backed by real `mmap()` memory, and extends into live process monitoring using platform-specific system calls. The city skyline visualization provides an intuitive and engaging representation of memory state.

Key contributions include: (1) a dual-mode interface combining educational simulation with real OS process data; (2) a process detail modal exposing threads, faults, and executable paths; (3) a system-wide memory pressure indicator with visual thresholds; and (4) a clean C-to-React architecture using POSIX sockets and JSON APIs without external dependencies.

Future work includes adding paging and segmentation simulations, implementing multi-threaded request handling in the HTTP server, supporting additional platforms (Windows via WinAPI), and adding process CPU usage metrics to the live skyline.

Acknowledgment

The author thanks the faculty of the Department of Computer Science for their guidance on operating systems concepts, and acknowledges the open-source community for tools including GCC, React, and Vite that made this project possible.

References

[1] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Hoboken, NJ: Pearson, 2018.

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[3] K. Robbins, "Web-Based Visualization Tools for Teaching Operating Systems," in *Proc. ACM SIGCSE Tech. Symp. on Computer Science Education*, 2019, pp. 284–290.

[4] H. Patel, "htop — An Interactive Process Viewer for Unix," 2004. [Online]. Available: https://htop.dev/

[5] N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation*, 2007, pp. 89–100.

[6] Apple Inc., "libproc — Process Information API," macOS Developer Documentation. [Online]. Available: https://developer.apple.com/documentation

[7] The Linux Kernel Documentation, "The /proc Filesystem," 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/filesystems/proc.html

[8] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

[9] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA: O'Reilly, 2005.

[10] Meta Platforms Inc., "React — A JavaScript Library for Building User Interfaces," 2024. [Online]. Available: https://react.dev/

[11] E. You, "Vite — Next Generation Frontend Tooling," 2024. [Online]. Available: https://vitejs.dev/

[12] IEEE Std. POSIX.1-2017, "Standard for Information Technology — Portable Operating System Interface (POSIX)," IEEE, 2017.